

Anno Accademico 2024–2025

Metodi del Calcolo Scientifico – Progetto 2

Compressione di immagini tramite la DCT

Bonfanti Davide

Matricola: 873293

Contents

1	Obiettivo della prima parte	2
2	teoria	2
2.1	La Trasformata Discreta del Coseno	2
2.2	La Trasformata DCT2 per immagini	2
2.3	Significato dei coefficienti	3
2.4	Importanza della DCT nella compressione	3
3	Implementazione della DCT2	4
3.1	Costruzione della base ortogonale	4
3.2	DCT monodimensionale	4
3.3	DCT bidimensionale (DCT2)	5
4	confronto dei tempi	5
5	Grafico dei risultati	6
6	Analisi dei risultati	6
7	Seconda parte	7
7.1	compressione di immagini in toni di grigio	7
7.2	Struttura del software	7
7.3	Interfaccia utente	7
7.4	Verifica della correttezza	8

1 Obiettivo della prima parte

Lo scopo della prima parte del progetto è quello di implementare manualmente la trasformata discreta del coseno bidimensionale (DCT2) e confrontare i tempi di esecuzione con una versione ottimizzata fornita da una libreria (in questo caso, `scipy.fft.dctn` di SciPy). Il confronto si basa sull'esecuzione su matrici quadrate di dimensioni crescenti e mira a evidenziare le differenze di complessità computazionale: teoricamente $\mathcal{O}(N^3)$ per l'implementazione fatta in casa, e $\mathcal{O}(N^2 \log N)$ per la versione fast.

2 teoria

2.1 La Trasformata Discreta del Coseno

La Discrete Cosine Transform (DCT) è una trasformata lineare utilizzata per rappresentare un segnale come somma pesata di funzioni coseno a frequenze diverse. In particolare, la DCT è molto efficace nel compattare l'energia del segnale nei primi coefficienti, caratteristica che la rende particolarmente adatta per la compressione di immagini e segnali.

La DCT per un vettore x di lunghezza N è definita come:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot \cos \left[\frac{\pi}{N} \left(n + \frac{1}{2} \right) k \right], \quad k = 0, 1, \dots, N-1$$

Questa trasformata è reale e ortogonale, e i coefficienti X_k rappresentano il contenuto in frequenza del segnale. I valori di k bassi rappresentano frequenze basse (variazioni lente), mentre i valori alti rappresentano frequenze alte (variazioni rapide).

2.2 La Trasformata DCT2 per immagini

Nel contesto delle immagini, la DCT viene estesa in due dimensioni per agire su blocchi bidimensionali di pixel. L'immagine viene suddivisa in blocchi $F \times F$ e su ciascun blocco viene applicata la DCT2 definita come:

$$X_{k,\ell} = \sum_{i=0}^{F-1} \sum_{j=0}^{F-1} x_{i,j} \cdot \cos \left[\frac{\pi}{F} \left(i + \frac{1}{2} \right) k \right] \cdot \cos \left[\frac{\pi}{F} \left(j + \frac{1}{2} \right) \ell \right]$$

con $k, \ell = 0, \dots, F-1$. Il risultato è una matrice di coefficienti in cui la componente in alto a sinistra ($X_{0,0}$) è il valore medio, mentre le altre rappresentano frequenze crescenti in direzione orizzontale, verticale e diagonale.

2.3 Significato dei coefficienti

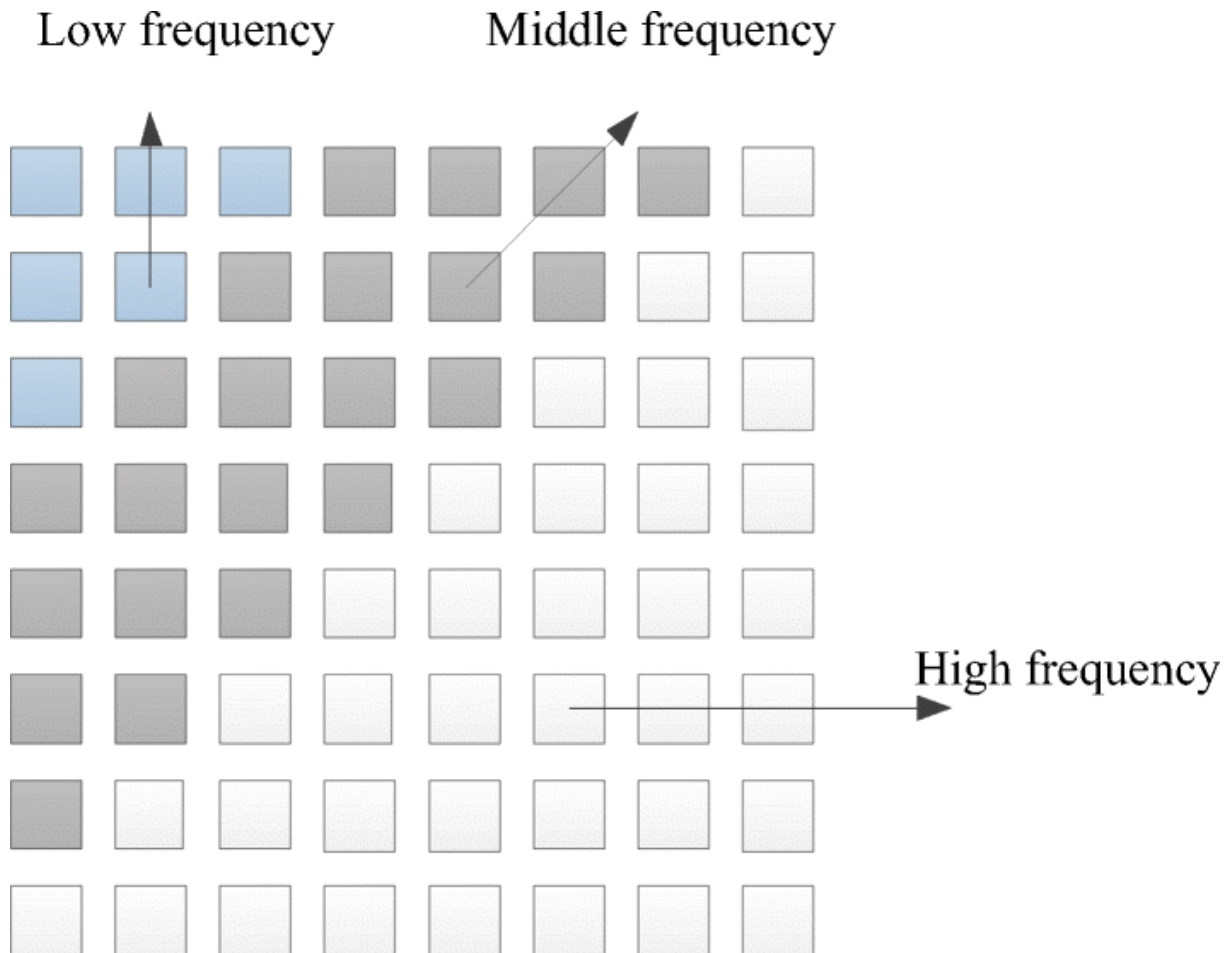


Figure 1: Enter Caption

Figure 2: Distribuzione spaziale delle frequenze nella DCT2.

Come mostrato in Figura 2, i coefficienti DCT2 rappresentano diverse componenti spaziali dell'immagine:

- Coefficienti vicini all'origine: frequenze basse, che rappresentano le variazioni più dolci e globali.
- Coefficienti lontani dall'origine: frequenze alte, legate ai dettagli fini, come contorni netti o rumore.

La maggior parte dell'informazione visiva è contenuta nei coefficienti a bassa frequenza, per cui è possibile scartare quelli ad alta frequenza senza compromettere troppo la qualità visiva.

2.4 Importanza della DCT nella compressione

La DCT è utilizzata in standard come JPEG proprio per la sua capacità di separare le componenti essenziali del segnale da quelle meno rilevanti. Dopo la trasformata, si

possono:

1. Eliminare i coefficienti più piccoli.
2. Quantizzare i rimanenti.
3. Codificarli in modo efficiente.

In particolare, l'eliminazione dei coefficienti ad alta frequenza è giustificata dal fatto che l'occhio umano è meno sensibile ai dettagli fini, specialmente nelle alte frequenze.

3 Implementazione della DCT2

Per la compressione delle immagini in toni di grigio è stata realizzata un'implementazione personalizzata della trasformata discreta del coseno (DCT) bidimensionale, seguendo la definizione teorica e senza fare uso di algoritmi ottimizzati basati su FFT. La trasformata è stata applicata su blocchi quadrati $F \times F$ dell'immagine.

3.1 Costruzione della base ortogonale

La DCT è stata implementata costruendo esplicitamente una matrice base ortogonale $D \in \mathbb{R}^{F \times F}$, definita nel seguente modo:

$$D_{k,j} = \alpha(k) \cdot \cos\left(\frac{\pi}{F} \left(j + \frac{1}{2}\right) k\right) \quad \text{per } k, j = 0, \dots, F-1$$

dove:

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{F}} & \text{se } k = 0 \\ \sqrt{\frac{2}{F}} & \text{se } k > 0 \end{cases}$$

Questa costruzione garantisce che la matrice D sia ortogonale, quindi l'inversa coincide con la trasposta: $D^{-1} = D^T$.

```
def dct_base(f):
    D = np.zeros((f, f))
    for i in range(f):
        for j in range(f):
            if i == 0:
                D[i, j] = 1 / np.sqrt(f)
            else:
                D[i, j] = np.sqrt(2 / f) * np.cos((np.pi * (2 * j + 1) *
                    i) / (2 * f))
    return D
```

Listing 1: Costruzione della matrice base DCT

3.2 DCT monodimensionale

Data una sequenza f di lunghezza F , la DCT e la sua inversa (IDCT) si ottengono semplicemente tramite moltiplicazione con la matrice D :

```
def dct(f, D):  
    return D @ f  
  
def idct(c, D):  
    return D.T @ c
```

Listing 2: DCT e IDCT monodimensionali

3.3 DCT bidimensionale (DCT2)

Nel caso bidimensionale, la trasformata DCT2 si ottiene applicando la DCT monodimensionale prima per riga e poi per colonna, sfruttando la stessa matrice base D :

$$B' = D \cdot B \cdot D^T \quad \text{e} \quad \hat{B} = D^T \cdot B' \cdot D$$

dove B è il blocco originale e B' la sua trasformata.

```
def dct2(block, D):  
    return D @ block @ D.T  
  
def idct2(block, D):  
    return D.T @ block @ D
```

Listing 3: Implementazione della DCT2 e della IDCT2

4 confronto dei tempi

Le matrici utilizzate sono quadrate di dimensioni variabili:

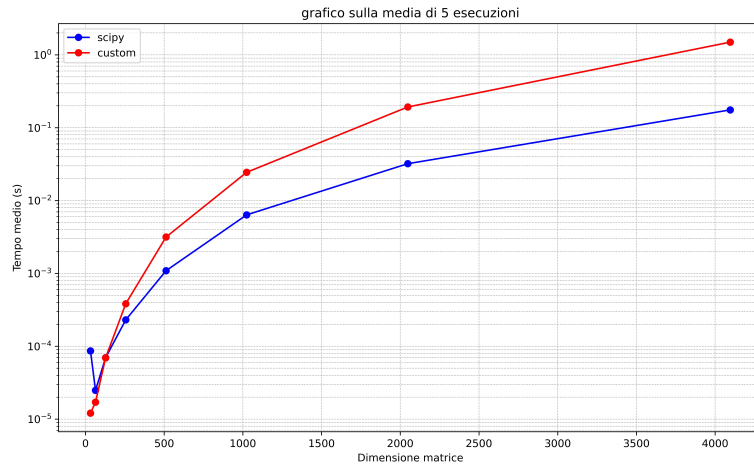
$N = 32, 64, 128, 256, 512, 1024, 2048, 4096$.

Per ogni dimensione:

- Si genera una matrice casuale con valori float32.
- Si applica la DCT2, si annulla una parte delle alte frequenze.
- Si ricostruisce la matrice con la IDCT2.
- Si misura il tempo medio di esecuzione su 5 ripetizioni.

5 Grafico dei risultati

Il grafico riporta i tempi medi di esecuzione delle due versioni della DCT2 al variare di N .



6 Analisi dei risultati

Come ci si aspettava dalla teoria, i risultati ottenuti durante l'esperimento confermano chiaramente la differenza di complessità tra le due versioni della DCT2. L'implementazione custom mostra tempi di esecuzione che crescono molto rapidamente con l'aumentare della dimensione N della matrice. Questo comportamento è coerente con una complessità dell'ordine di $\mathcal{O}(N^3)$. Al contrario, l'implementazione con SciPy, che utilizza la funzione `dctn` del modulo `scipy.fft`, risulta essere molto più efficiente con una complessità dell'ordine di $\mathcal{O}(N^2 \log N)$.

7 Seconda parte

7.1 compressione di immagini in toni di grigio

In questa seconda parte del progetto si realizza un software open-source per la compressione di immagini in toni di grigio, in formato BMP, utilizzando la DCT2 su blocchi quadrati di dimensione $F \times F$. Viene applicata una soglia d per azzerare i coefficienti di alta frequenza al fine di ridurre l'informazione da memorizzare.

7.2 Struttura del software

Il software 'e scritto in Python e Flask per la parte web. ed 'e strutturato in tre moduli:

- **dct_utils.py**: contiene le funzioni per la costruzione della base DCT2 e per l'applicazione della DCT2 e IDCT2 a singoli blocchi.
- **app.py**: crea un server web con Flask che permette di caricare un'immagine in scala di grigi, applicare la compressione DCT2 a blocchi con soglia sulle frequenze, e mostra sia l'immagine originale che quella compressa direttamente nella pagina web. Gestisce la ricezione del file e dei parametri via form POST, elabora l'immagine usando le funzioni DCT e IDCT, e converte le immagini in base64 per visualizzarle nell'HTML.
- **index.html**: Crea una pagina web dove l'utente può caricare un'immagine .bmp, selezionare la dimensione dei blocchi e la soglia di taglio delle frequenze tramite slider, e vedere le immagini originale e compressa.

7.3 Interfaccia utente

L'interfaccia grafica permette di:

- Selezionare un'immagine BMP
- Impostare la dimensione dei blocchi F
- Impostare la soglia d per la compressione
- Visualizzare l'immagine compressa a confronto con l'originale

PROGETTO METODO DEL CALCOLO SCIENTIFICO

Carica un'immagine .bmp e scegli i parametri

1. Scegli l'immagine

Click per caricare
.BMP

2. Ampiezza delle finestrelle

seleziona la dimensione di F: 4

3. Soglia di taglio delle frequenze

valore: 0

Processa Immagine

Figure 3: Interfaccia grafica del software di compressione DCT2

7.4 Verifica della correttezza

Per verificare la correttezza dell'implementazione degli algoritmi DCT e DCT2 sono stati eseguiti due test.

Il primo test ha applicato la DCT a un vettore di 8 valori:

$$[231, 32, 233, 161, 24, 71, 140, 245]$$

Il risultato ottenuto è stato confrontato con una trasformata attesa nota:

$$[4.01 \times 10^2, 6.60, 1.09 \times 10^2, -1.12 \times 10^2, 6.54 \times 10^1, 1.21 \times 10^2, 1.16 \times 10^2, 2.88 \times 10^1]$$

Analogamente, nel secondo test è stato considerato un blocco 8×8 di valori in scala di grigi. L'algoritmo DCT2 è stato applicato al blocco, e i coefficienti ottenuti sono stati confrontati con una matrice attesa fornita nel testo del progetto.

Risultati Entrambi i test sono stati superati:

- **Test DCT:** errore relativo massimo pari a 7.01×10^{-3}
- **Test DCT2:** errore relativo massimo pari a 7.88×10^{-3}

Questi risultati confermano la correttezza numerica dell'implementazione sia per la trasformata monodimensionale che per quella bidimensionale.